

FunnelQ

Turn a marketing agency's raw funnel data into a deployed, decision-ready intelligence product — built with gradient boosting, shipped on real infrastructure, and locked behind a real login. Build it for yourself: no submission, no deadline, no grade.

XGBoost

LightGBM

CatBoost

GitHub

Supabase

Supabase Auth

Railway

Level: Intermediate **Est. effort:** 15–25 hours, at your own pace

Type: Self-directed · nothing to hand in **Dataset:** funnel_marketing_data.csv

WHAT'S IN THIS BRIEF

1. The scenario
2. Your mission
3. The dataset
4. The three pillars
5. The analytical work
6. What you'll build
7. Principles & good habits
8. Resources

01 The scenario

BACKGROUND

Northbound Media is a 40-person performance-marketing agency. For two years they've tracked every step of their sales funnel: the ad money spent, the leads it generated, how many picked up the phone, how many survived each follow-up call, which deals closed, how long each client stayed, whether they bought more, and whether they sent a referral.

All of it lives in a single spreadsheet nobody fully trusts.

Last quarter, Northbound's founder sat down with the growth team after another six-figure ad month and said the quiet part out loud:

"We're spending a fortune on ads, but I genuinely can't tell you what makes a customer stay, spend more, or refer a friend. We're optimizing on gut feel. Sales wants to cut follow-ups after the third call. Finance wants to slash the ad budget. Nobody has evidence. I need us to stop guessing and start knowing — and I need it as a tool the team can log into and actually use, not a slide from a data scientist that we forget in a week."

That last sentence is the whole project. A notebook full of charts is *not* the goal. The founder needs a living product: an internal tool the team signs into, with data in a real database, models behind a real API, a dashboard the team can open in a browser, all running on infrastructure that stays up after you close your laptop. That is exactly the end-to-end a junior ML / data engineer is hired to own — and exactly what makes this a portfolio piece instead of a homework set.

WHY THIS MATTERS FOR YOUR PORTFOLIO

Anyone can fit a model in a notebook. What gets you hired is proving you can take a messy business problem all the way to something deployed, secured behind a login, versioned, and usable by non-technical people. When you're done you'll have a public GitHub repo, a live URL, and a story you can tell in an interview.

02 Your mission

Build and deploy **FunnellQ** — a marketing-intelligence tool for Northbound Media's team.

By the end, FunnellQ must be able to answer the founder's real questions:

- **How long will a new customer stay?** (predict lifetime in months)
- **Who is likely to buy more?** (predict upsell probability)
- **Who will become a "super customer"** — someone who stays, spends, and refers? (a 0–100 score)
- **Where should the ad budget go** to maximize expected profit?
- **Are late follow-ups really a waste of time**, or is the sales manager wrong?

And because this is an internal tool, all of it sits behind a **login screen** — only Northbound's team can get in. Each answer must be reachable through your deployed, signed-in application — not just printed once in a notebook cell. The [analytical work](#) section breaks these down into concrete requirements.

03 The dataset

You are given `funnel_marketing_data.csv` — one row per customer/campaign record. It is real-world messy: expect missing values and relationships that aren't as clean as you'd hope. Treat data cleaning as part of the job, not a nuisance.

Column	Description
<code>ad_budget</code>	Monthly advertising spend (₺)
<code>num_leads</code>	Total leads generated
<code>leads_answered</code>	Leads that picked up the phone
<code>leads_not_answered</code>	Leads that didn't answer
<code>followup_1</code> ... <code>followup_5</code>	Leads remaining after each follow-up round
<code>not_closed</code>	Leads that went through the funnel but didn't convert
<code>closed</code>	Number of deals closed
<code>calls_to_closed</code>	Average calls before closing a deal
<code>calls_to_not_closed</code>	Average calls before giving up
<code>customer_acquisition_cost</code>	Cost per acquired customer (₺)
<code>ltv_months</code>	Customer lifetime in months
<code>purchased</code>	Made an initial purchase? (1/0)
<code>upsell</code>	Bought additional services? (1/0)
<code>cumulative_profit</code>	Total profit generated from the customer (₺)
<code>referred</code>	Did the customer refer someone? (Yes/No)

THINK BEFORE YOU MODEL

Some columns describe what happened *after* a customer's relationship played out (for example, total profit earned). Using an outcome to predict another outcome is **data leakage** and will make your model look brilliant in training and useless in production. The real skill here is deciding, per task, which features a business would actually have available at prediction time — and being able to defend that choice.

04 The three pillars

FunnellIQ is not just a model — it's a shipped system. Three infrastructure pillars are the backbone. Your architecture is up to you, but each pillar should be genuinely used (a screenshot of a signup page is not "used").



GitHub

Source of truth, collaboration, and automation live here.



Supabase

Postgres for the dataset *and* Auth for the login screen.



Railway

Your service runs on a public URL that stays up.

GitHub — version control & automation

- A public repository with a clear, professional `README.md` (project overview, architecture diagram or description, local setup steps, and the live URL).
- A **meaningful commit history** — small, descriptive commits, not one giant "final" dump.
- Use **feature branches** and open at least one **Pull Request** (even solo — it shows you know the workflow).
- A `.gitignore` that keeps secrets, data dumps, and virtual-environments out of the repo. **No API keys or `.env` files may ever be committed.**
- A **working GitHub Actions workflow that runs on every push** (at minimum: install dependencies and run your tests/lint).

Supabase — data & authentication

Data layer:

- Provision a Supabase project and design a table (or tables) for the dataset. Include a schema file (`schema.sql`) in the repo.
- Load the CSV into the database with a repeatable script — not by hand.
- Your application must **read from Supabase at runtime** (for at least one feature, e.g. serving historical records or computing insights), using credentials from environment variables.

Authentication — the login screen:

- **Build a login screen using Supabase Auth** (email + password at minimum). FunnellIQ is an internal tool for Northbound's team, so the app should sit behind a sign-in rather than being open to the world.

- **Handle the session properly:** an unauthenticated visitor sees only the login screen; a signed-in user reaches the dashboard and predictions; **sign-out works** and clears the session.
- Use the **public anon key** in the browser for the login flow — never expose the service key client-side. Keep any privileged operations server-side.
- Tie it together with access control: enable **Row Level Security** and write policies so the database itself enforces that only authenticated users can read the data.

Railway — deployment

- Deploy your service to Railway so it's reachable at a public URL.
- Secrets (Supabase keys, config) are set as **Railway environment variables**, never hard-coded.
- Connect the Railway service to your GitHub repo so pushes trigger a redeploy.
- Expose a health-check endpoint and confirm the deployment survives a restart.

SUGGESTED (NOT REQUIRED) STACK

A Python API (FastAPI or Flask) serving JSON endpoints plus a lightweight HTML/JS dashboard is a clean fit and deploys to Railway easily. Supabase's JavaScript auth client makes the login screen a small amount of front-end code — but you may use any stack you like. What counts is that the three pillars work together, not which framework you picked.

05 The analytical work

Six work packages. Each one produces something your deployed FunnelIQ exposes (an endpoint, a dashboard panel, or a documented finding). What matters is **rigor and reasoning**, not hitting a magic accuracy number — an honest model with a clear-eyed evaluation beats an inflated one every time. Do them in order or cherry-pick the ones you want to practice.

Package 1 Exploration & cleaning

Understand the data before you trust it.

Build: a short written findings note (in the README or a `/docs` file) covering missing-value handling, a correlation analysis against `cumulative_profit`, the shape of the `ad_budget` → `num_leads` relationship, and conversion rate (`closed / num_leads`) across budget tiers — Low (≤ 1500), Mid (2000–5000), High (> 5000).

Questions to answer: How many rows are incomplete, and how did you handle them? Does more budget buy proportionally more leads, or do you see diminishing returns? Which budget tier converts best — and does that surprise you?

Package 2 Predict customer lifetime (regression)

Target: `ltv_months`. "Can we predict how long a customer will stay?"

Build: three trained models — XGBoost, LightGBM, and CatBoost — compared with 5-fold cross-validation on RMSE and R^2 . Extract and compare feature importances across all three. Expose the prediction through your app.

Questions to answer: Should `cumulative_profit` be a feature here? Justify it. Which features dominate, and do the three models agree? In two sentences, what's the strongest lever on customer longevity, and what should Northbound do about it?

Package 3 Predict upsell probability (classification)

Target: `upsell` . "Which customers are most likely to buy more?"

Build: three classifiers with **class-imbalance handling** (e.g. `scale_pos_weight` or class weights) and 5-fold **stratified CV**. Report Accuracy, Precision, Recall, and F1. Plot importances for your best model.

Questions to answer: Why is accuracy alone misleading here? Is upsell driven by one feature or a combination? Design a simple business rule (e.g. "if `LTV > X` and `CAC < Y`, flag for outreach") and compare it against your model's predictions — where does the rule win or lose?

Package 4 The "super customer" score

Target: `referred` . Super customers stay long, buy more, and bring in new clients.

Build: a CatBoost classifier (use its native categorical handling) with **hyperparameter tuning** (search over learning rate, depth, and iterations). Then build a scoring pipeline: given a new customer's early funnel data, output a **0–100 likelihood** of becoming a super customer, served by your app.

Questions to answer: Profile the super customers (`referred = Yes`, `upsell = 1`, long tenure): what share of total revenue do they represent, and what's their average acquisition cost? How could Northbound spot them earlier?

Package 5 The follow-up paradox

The sales manager claims: "After the 3rd follow-up, we're just wasting time." Is that true?

Build: the dropout rate at each follow-up stage (`followup_1` → `followup_5`), visualized, plus a data-driven recommendation surfaced in the dashboard.

Questions to answer: At which stage does dropout behave unexpectedly? For deals that eventually closed, how many follow-ups did they typically take? Should Northbound change its follow-up policy — yes or no, and why?

Package 6 Budget optimization challenge

Northbound has $\approx$ 50,000/month to spend and currently spreads it evenly.

Build: a model that predicts `cumulative_profit` , used to simulate allocation strategies — e.g. one big campaign vs. ten mid-size vs. many small — and a recommendation for which maximizes expected total profit. Expose the simulator in your app.

Questions to answer: Based on what your model learned about diminishing returns, does concentrating or spreading spend win? What would you tell the founder to do next month?

06 What you'll build

There's nothing to hand in — this is yours. But if you finish, you'll walk away with a set of artifacts that are genuinely worth showing off in an interview or on a résumé:

1. **A public GitHub repository** — code, `README.md` , `schema.sql` , a CI workflow, and a commit history that shows how you worked.
2. **A live application URL on Railway** — a real link you can share that actually answers the founder's questions.
3. **A login-gated dashboard** — a real Supabase Auth sign-in screen, not an open page, so the app behaves like the internal tool it's meant to be.
4. **A short write-up** (nice to keep as `REPORT.md`) capturing your findings and the business recommendations from the work packages — the part that proves you can turn models into decisions.
5. **A quick demo** — an optional 3–5 minute screen-recording of the deployed app in action. Future-you will thank present-you when it's time to talk about the project.

YOU'LL KNOW IT'S DONE WHEN...

A stranger can open your live URL, sign in, get a prediction for a new customer, see the follow-up and budget insights, and read your recommendations — without you touching anything. Your repo tells them how it was built and how to run it themselves. That's the bar worth aiming for, entirely for your own satisfaction.

07 Principles & good habits

No one's checking, but these are the habits that separate a toy project from one that looks professional — worth building in from the start.

- **Reproducibility.** Anyone (including future-you) should be able to clone the repo, follow the README, and stand it up. Pin your dependencies.
 - **Secrets stay secret.** Everything sensitive goes in environment variables and is excluded by `.gitignore` — never commit an API key or `.env`. This one's a real-world habit worth locking in now.
 - **Anon key in the browser, service key on the server.** The login screen uses Supabase's public anon key; the service-role key never leaves your backend. Getting this split right is the whole point of the auth exercise.
 - **No leakage without justification.** If you include a feature that looks like an outcome, write down why it's legitimately available at prediction time. This is the single most valuable modeling instinct in the whole project.
 - **Free tiers are plenty.** Supabase and Railway both have free tiers that comfortably cover this project. You don't need to spend a shekel.
 - **Use AI tools if you like — but understand what you ship.** The point is to be able to explain every line, because that's what makes it yours.
 - **Credit what you borrow.** Note snippets from docs or tutorials in comments or the README. Good habit, and it helps you find them again later.
-

08 Resources

- **Gradient boosting:** XGBoost, LightGBM, and CatBoost documentation (installation, defaults, feature importance, categorical handling).
- **Supabase data:** the official docs on creating a project, the SQL editor, table setup, Row Level Security, and the Python client.
- **Supabase Auth:** the docs on email/password sign-in, the JavaScript auth client, and managing user sessions — this is what powers your login screen.
- **Railway:** the deploy-from-GitHub guide, environment variables, and health checks.
- **API framework:** FastAPI or Flask quick-start guides if you go the Python route.
- **Git workflow:** any solid primer on feature branches, pull requests, and writing good commit messages.

ONE PIECE OF ADVICE

Get the boring skeleton deployed on day one — an empty app that returns "hello" from Railway, reading nothing but a health check. Deployment and auth problems are much easier to debug when there's almost no code. Then grow the project inside a pipeline that already works.